From Trees to Register Code

Guillermo Polito

guillermo.polito@inria.fr
@guillep













Goals

- Understanding code translation targeting register machines

- Tree traversals and execution semantics
- Translation as tree traversal

- Registers are limited!

Remember Stack-Based Bytecode

```
push 3

push a

push b

send +

send /

send ceiling
```

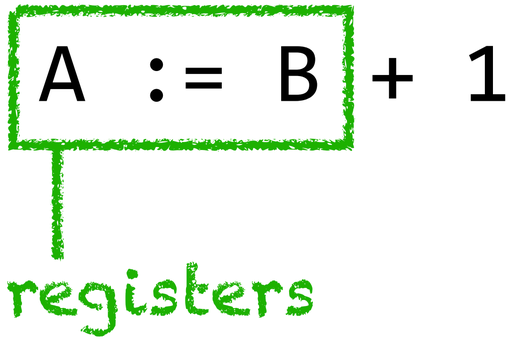
```
(3 / (a + b)) . ceiling()
```

- Each operation produces or consumes values into/from the stack
- Compact linear representation
- Execution “falls” from one instruction to the next one

Remember Register Machines

- Operands are exchanged through *explicit registers*

- Each machine defines its own set of registers
  - Could be fixed or *infinite*
  - Could be physical or virtual/symbolic



Stack and Register Mismatch

```
(3 / (a + b)) . ceiling()
```

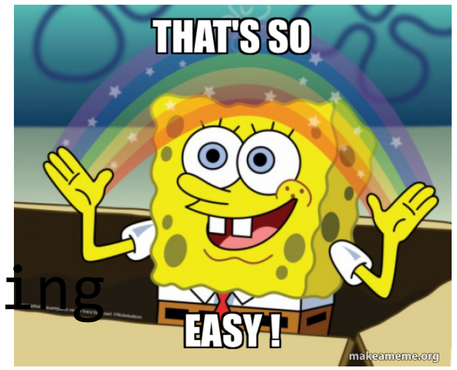
```
push 3
push a
push b
send +
send /
send ceiling
```

```
T1 := A + B
T2 := 3 / T1
T3 := T2 SEND ceiling
```

Stack and Register Mismatch

```
(3 / (a + b)) . ceiling()
```

```
push 3
push a
push b
send +
send /
send ceiling
```



Stack looks infinite!

How do we choose the registers?

```
T1 := A + B
T2 := 3 / T1
T3 := T2 SEND ceiling
```



Translation Insight

- One way to think about it: we simulate the stack-based execution.

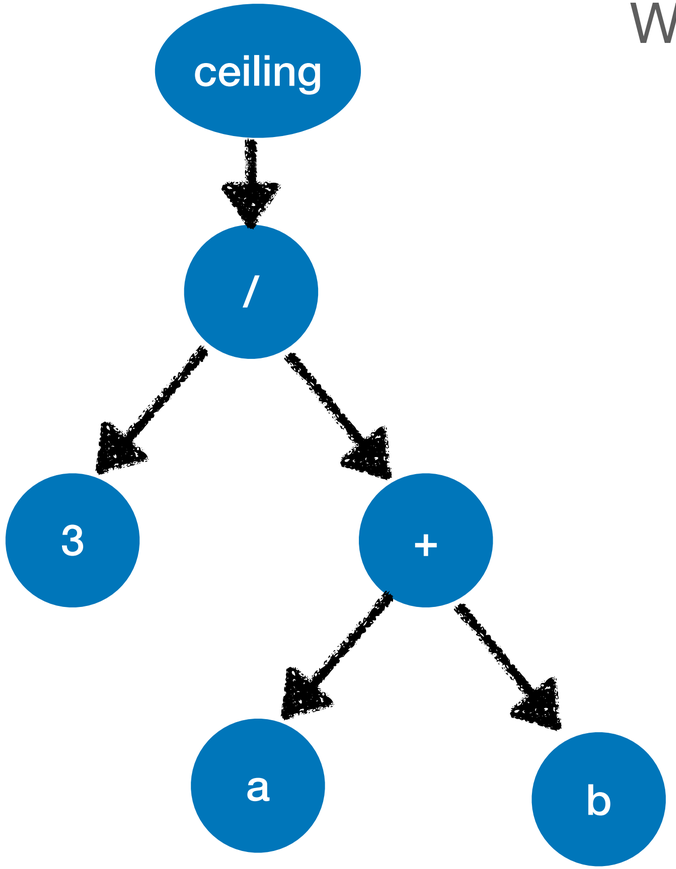
  =>Similar traversal!

- Each position in the stack is mapped to a register

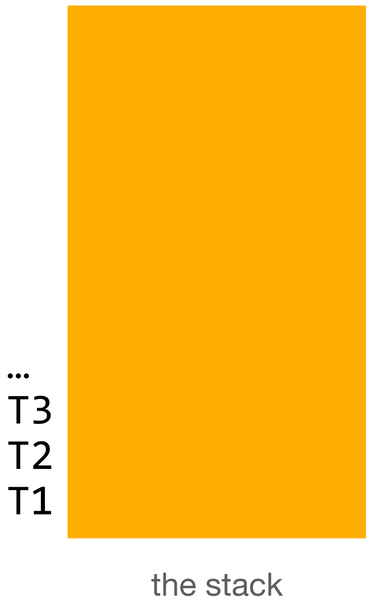
- ***Pushing*** to the stack creates ***sets values to registers***
- ***Popping*** from the stack gives us ***the register to use***

Translation Overview

```
(3 / (a + b)) . ceiling()
```



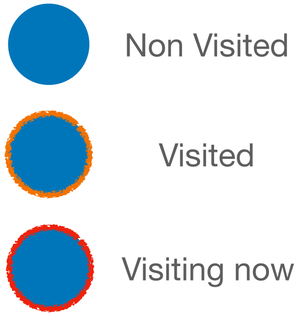
Depth-first post-order.
We traverse the tree bottom up.




```
T1 := 3
T2 := a
T3 := b
T2 := T2 + T3

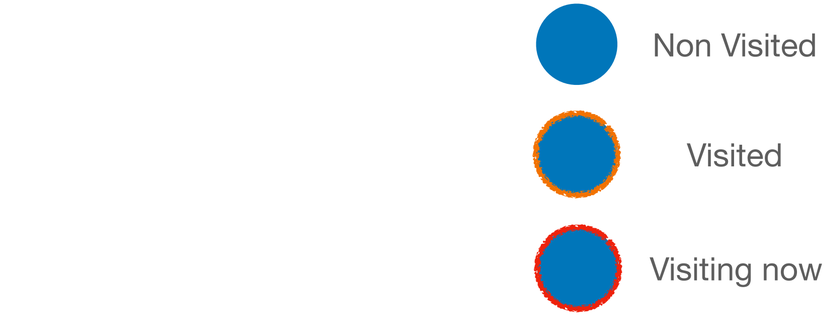
T1 := T1 / T2

T1 := T1 SEND ceiling
```

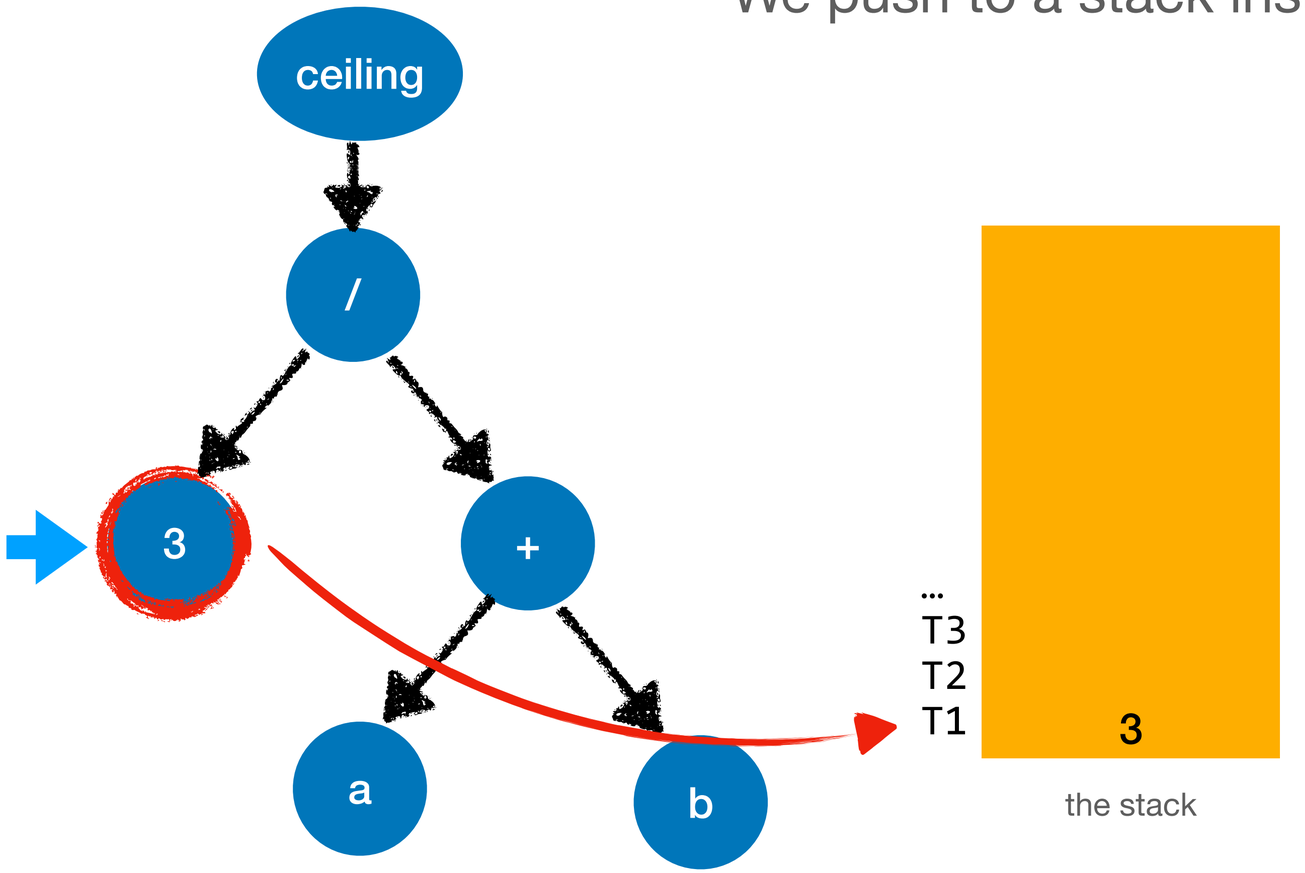


Translation Overview

```
(3 / (a + b)) . ceiling()
```



We push to a stack instead of generating a push



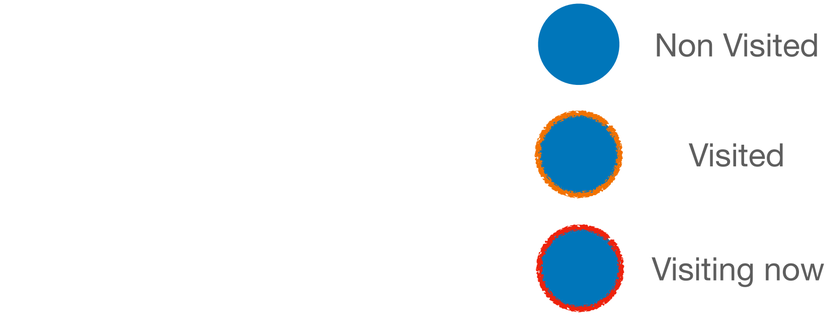

```
T1 := 3
T2 := a
T3 := b
T2 := T2 + T3

T1 := T1 / T2

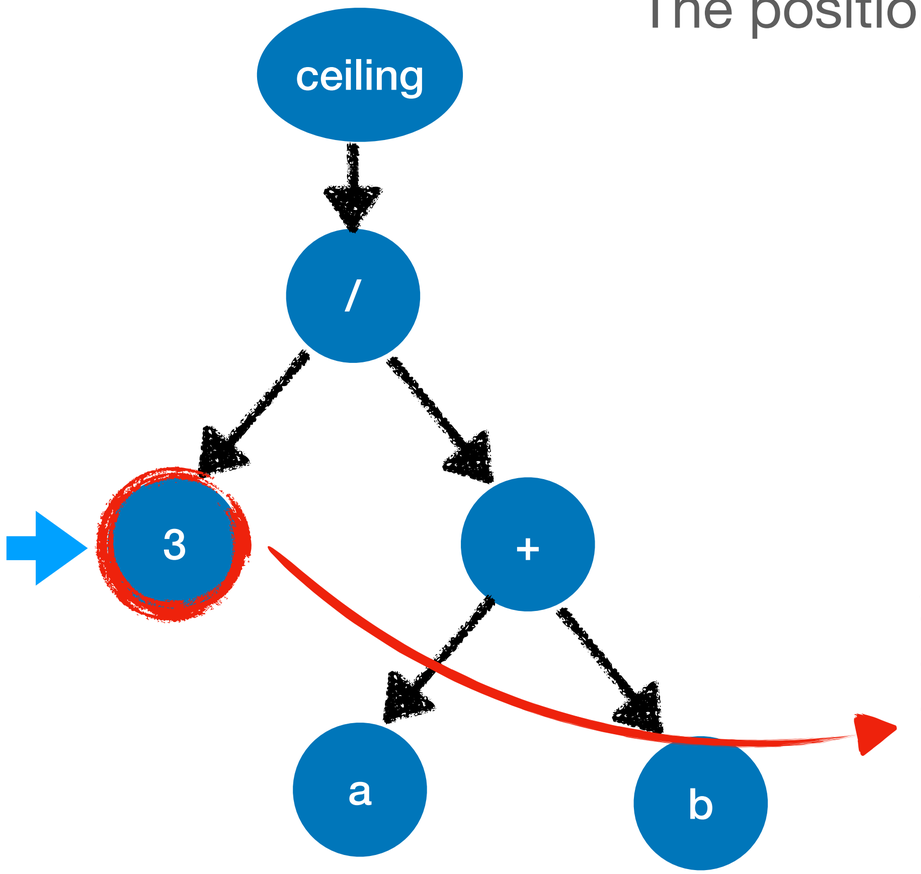
T1 := T1 SEND ceiling
```

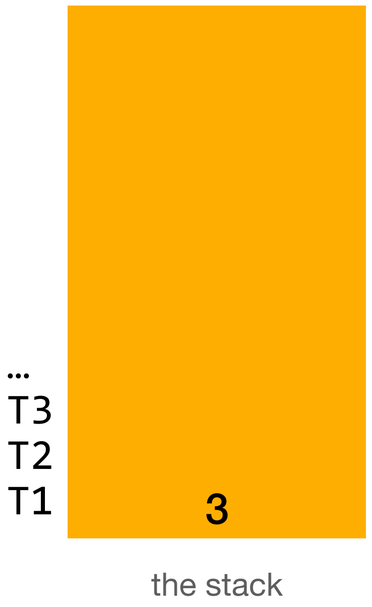
Translation Overview

```
(3 / (a + b)) . ceiling()
```



The position gives the destination register





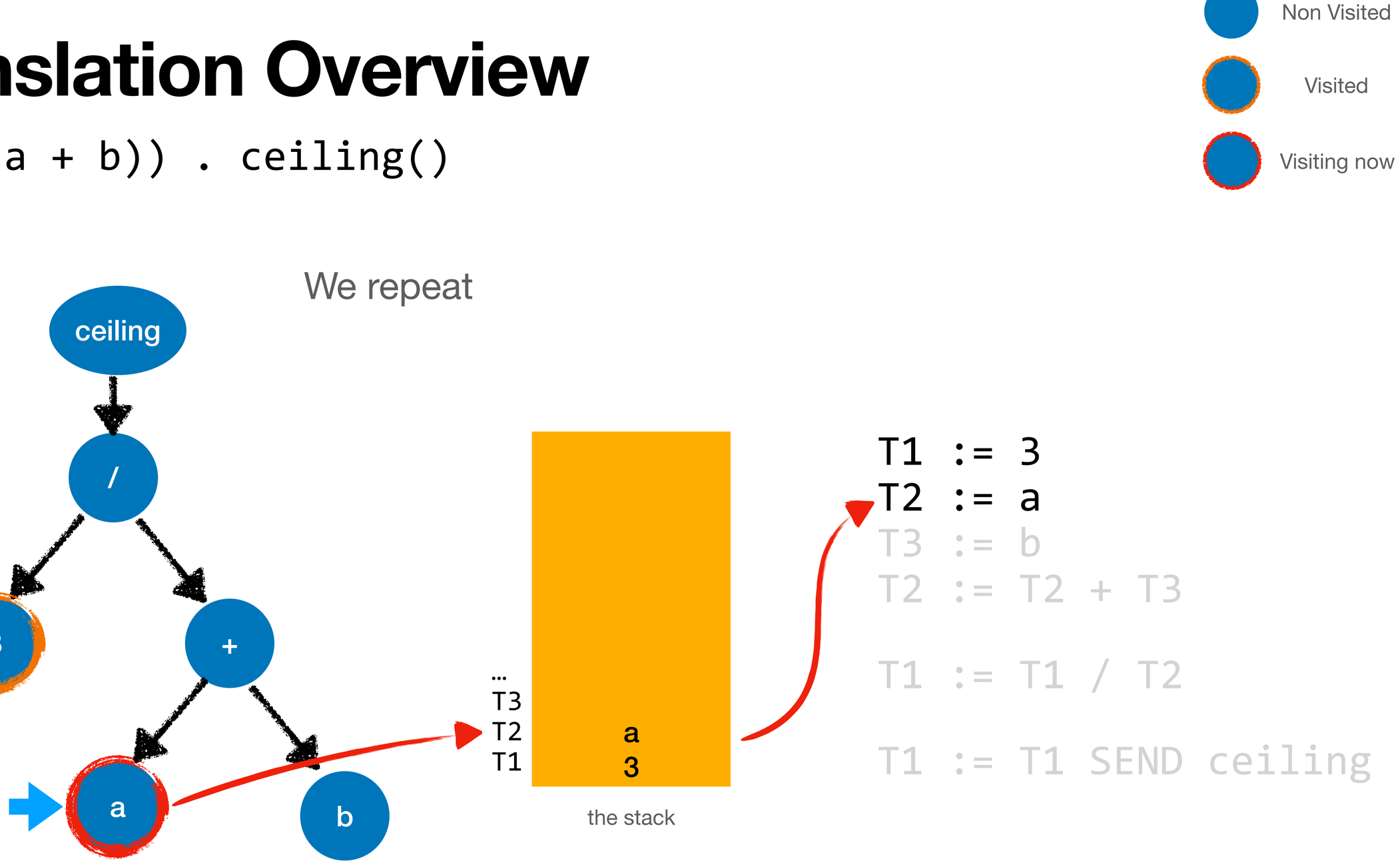

```
T1 := 3
T2 := a
T3 := b
T2 := T2 + T3

T1 := T1 / T2

T1 := T1 SEND ceiling
```

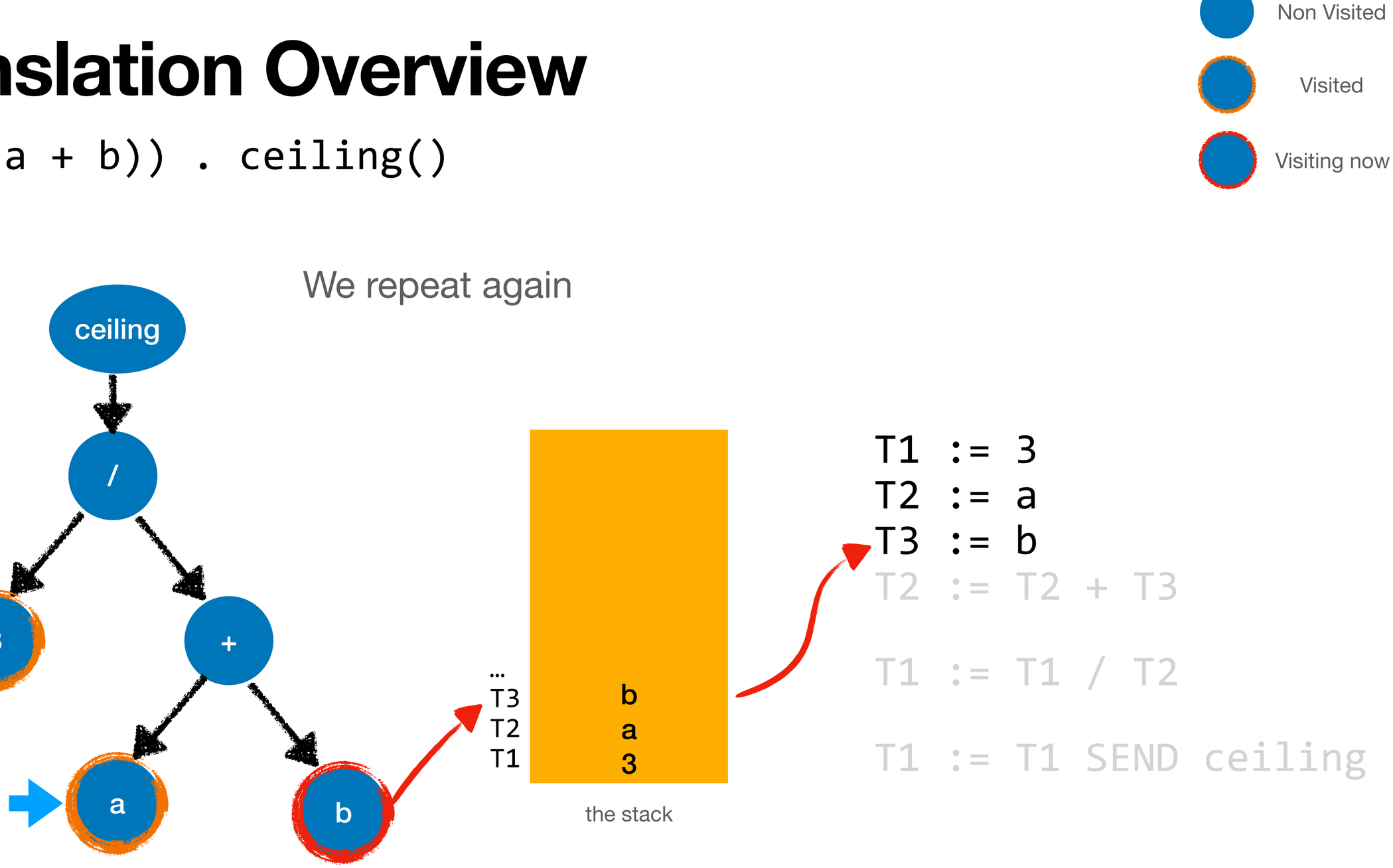
Translation Overview

```
(3 / (a + b)) . ceiling()
```



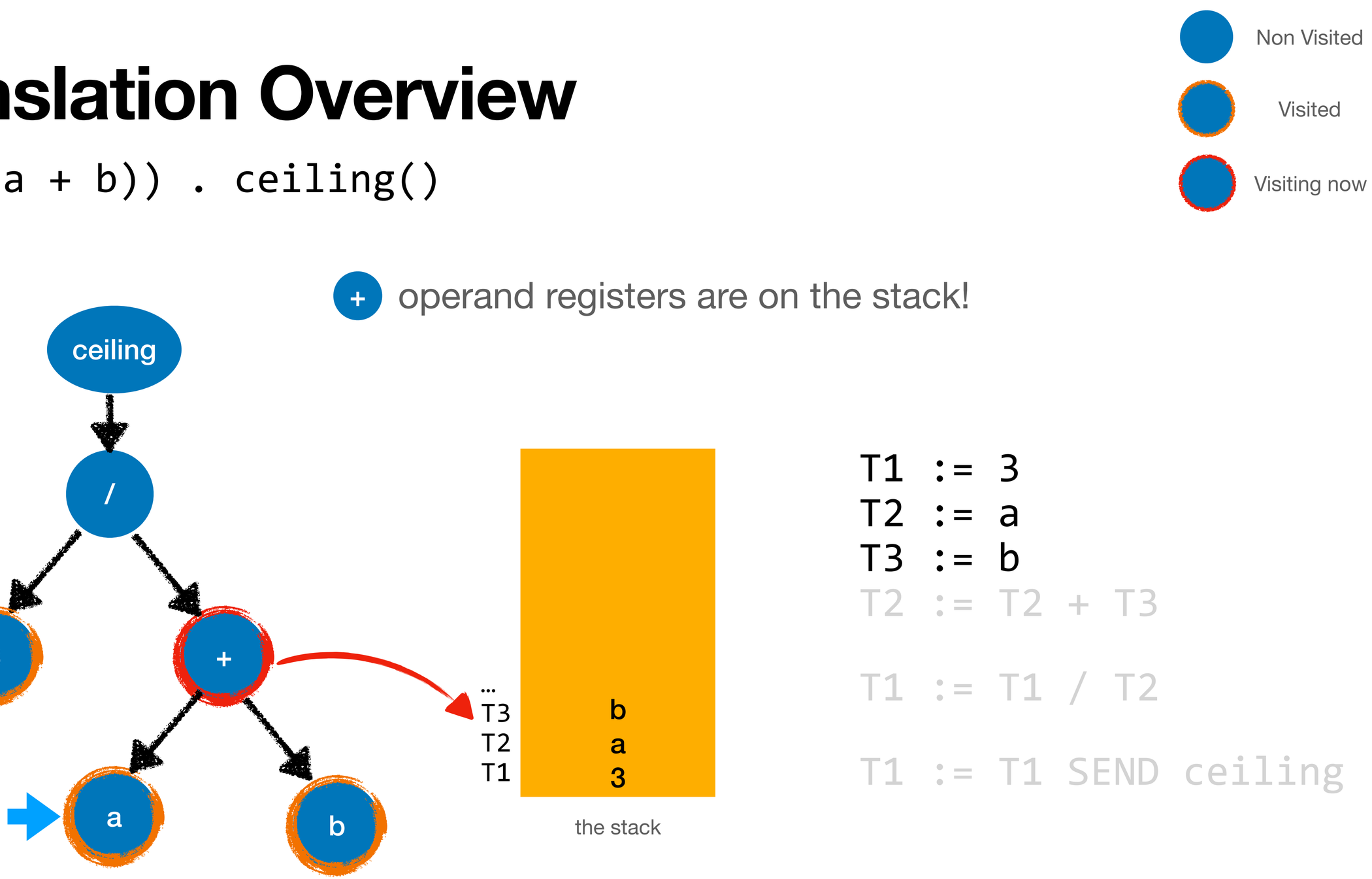
Translation Overview

```
(3 / (a + b)) . ceiling()
```



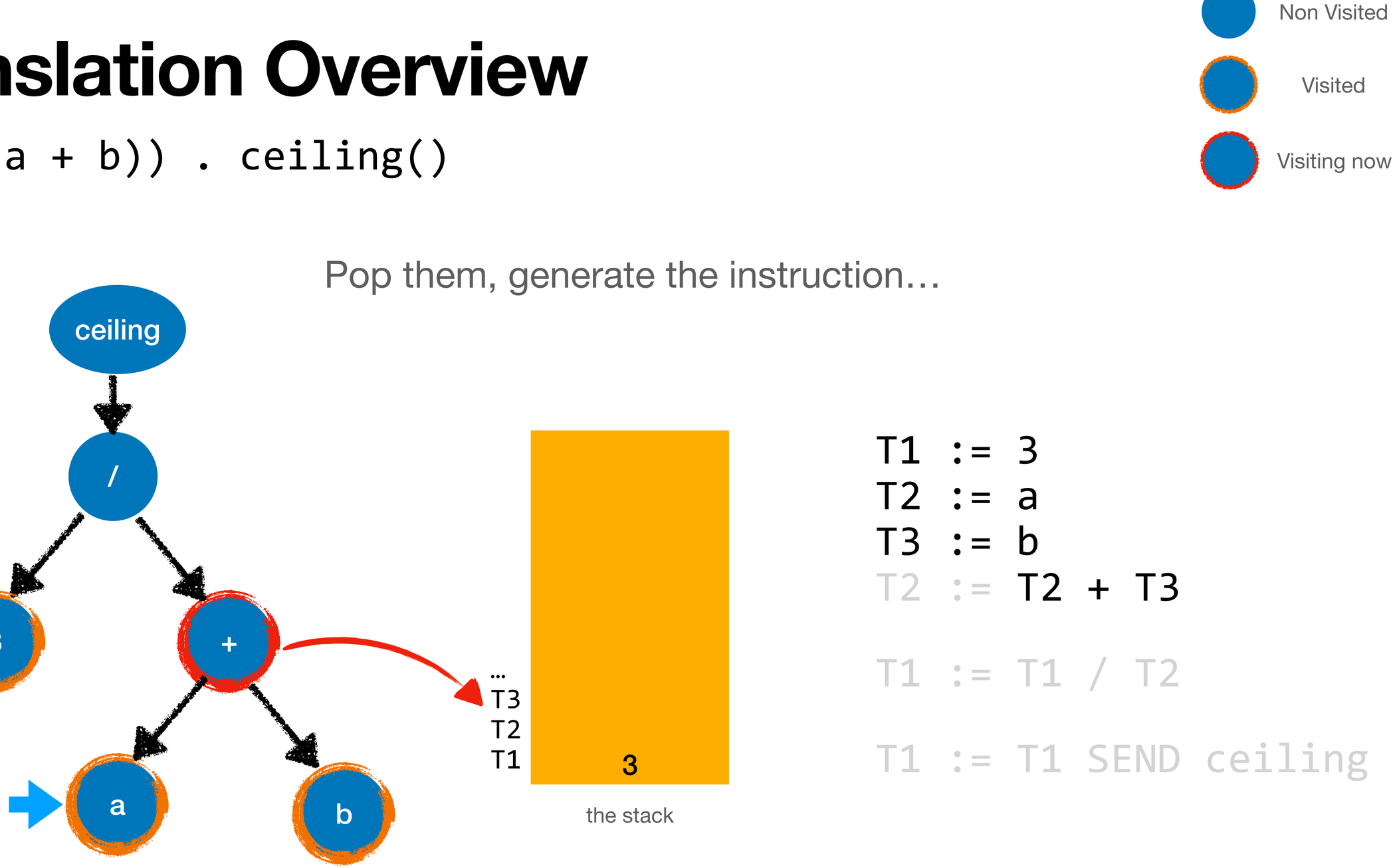
Translation Overview

```
(3 / (a + b)) . ceiling()
```



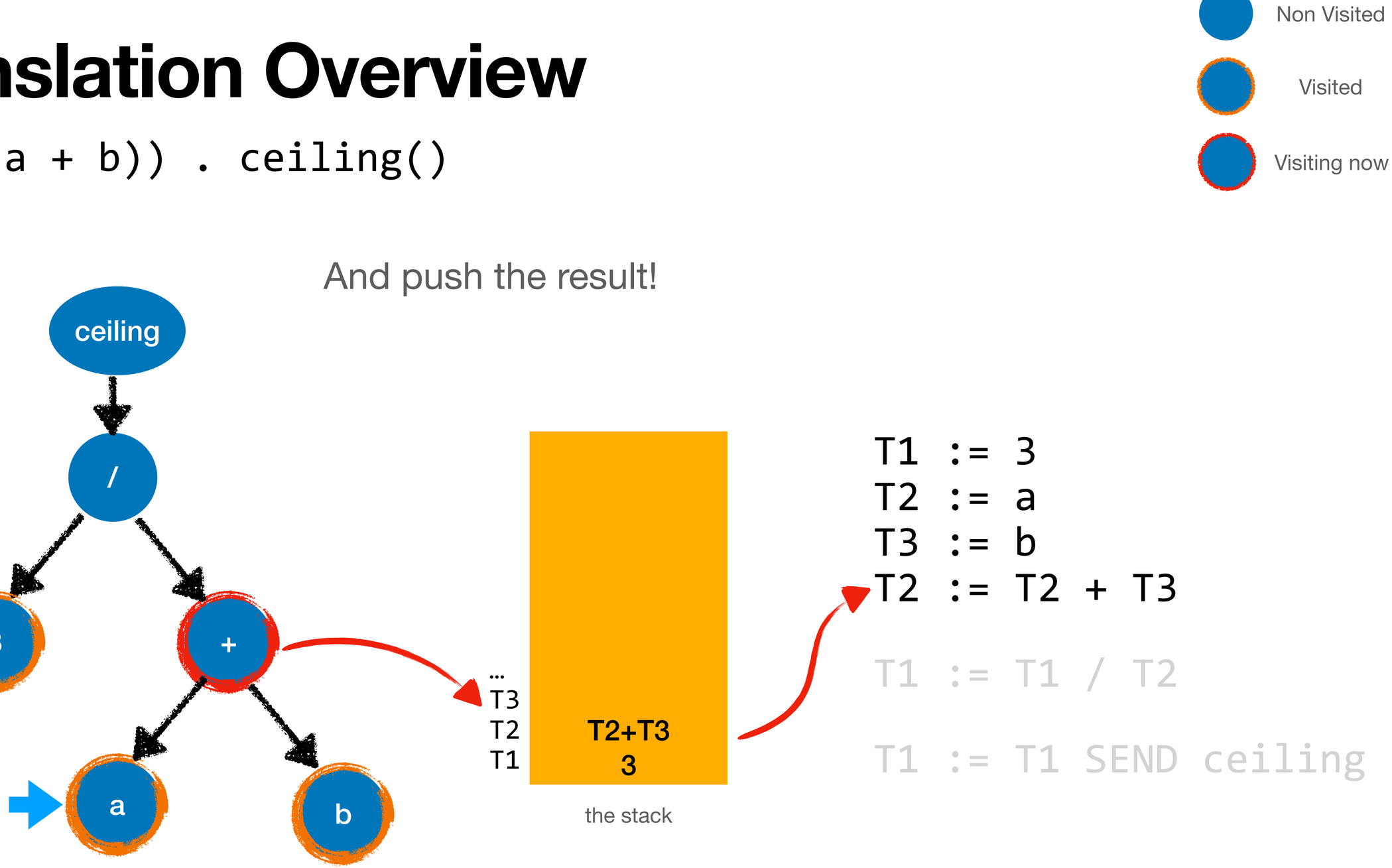
Translation Overview

```
(3 / (a + b)) . ceiling()
```



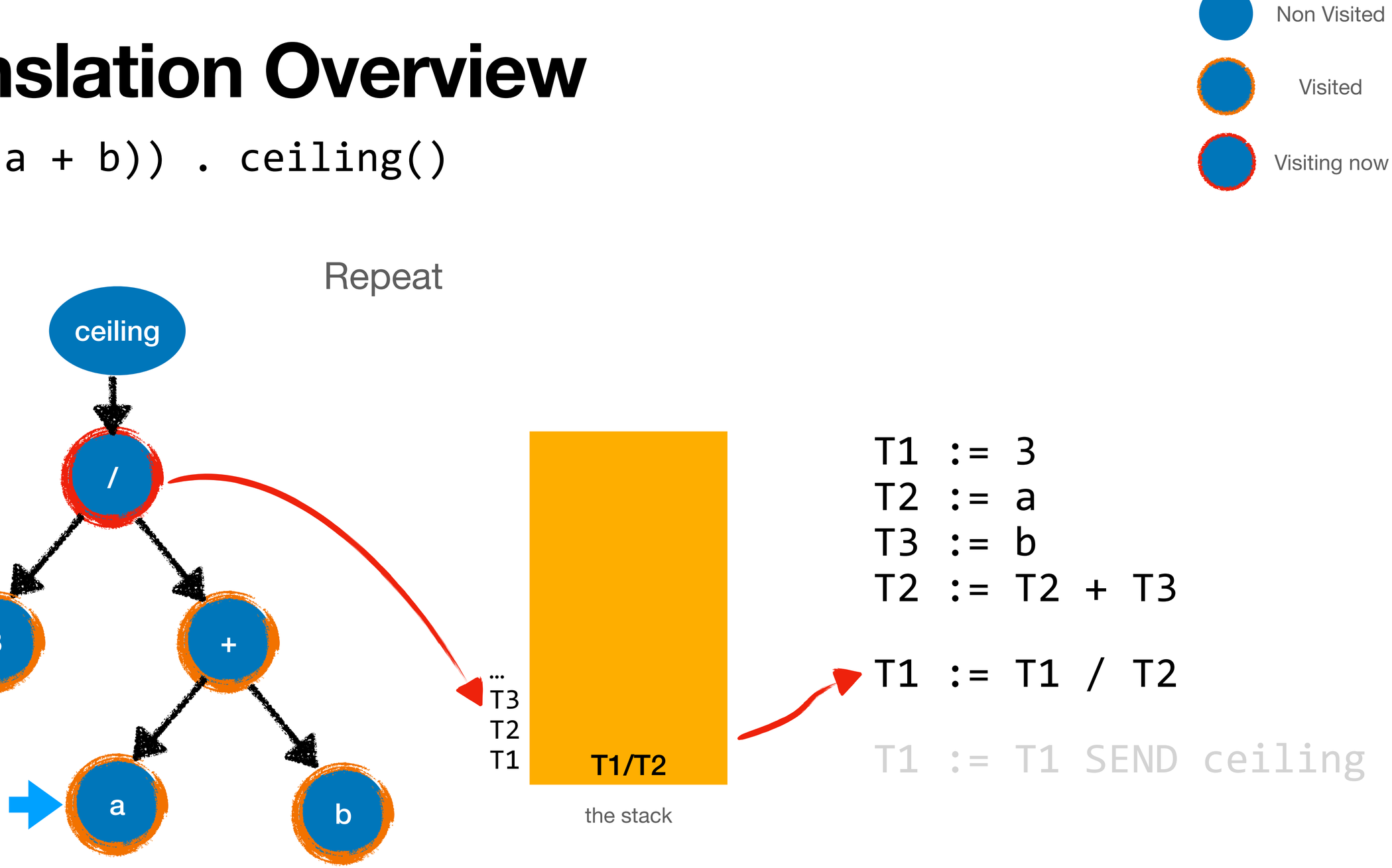
Translation Overview

```
(3 / (a + b)) . ceiling()
```



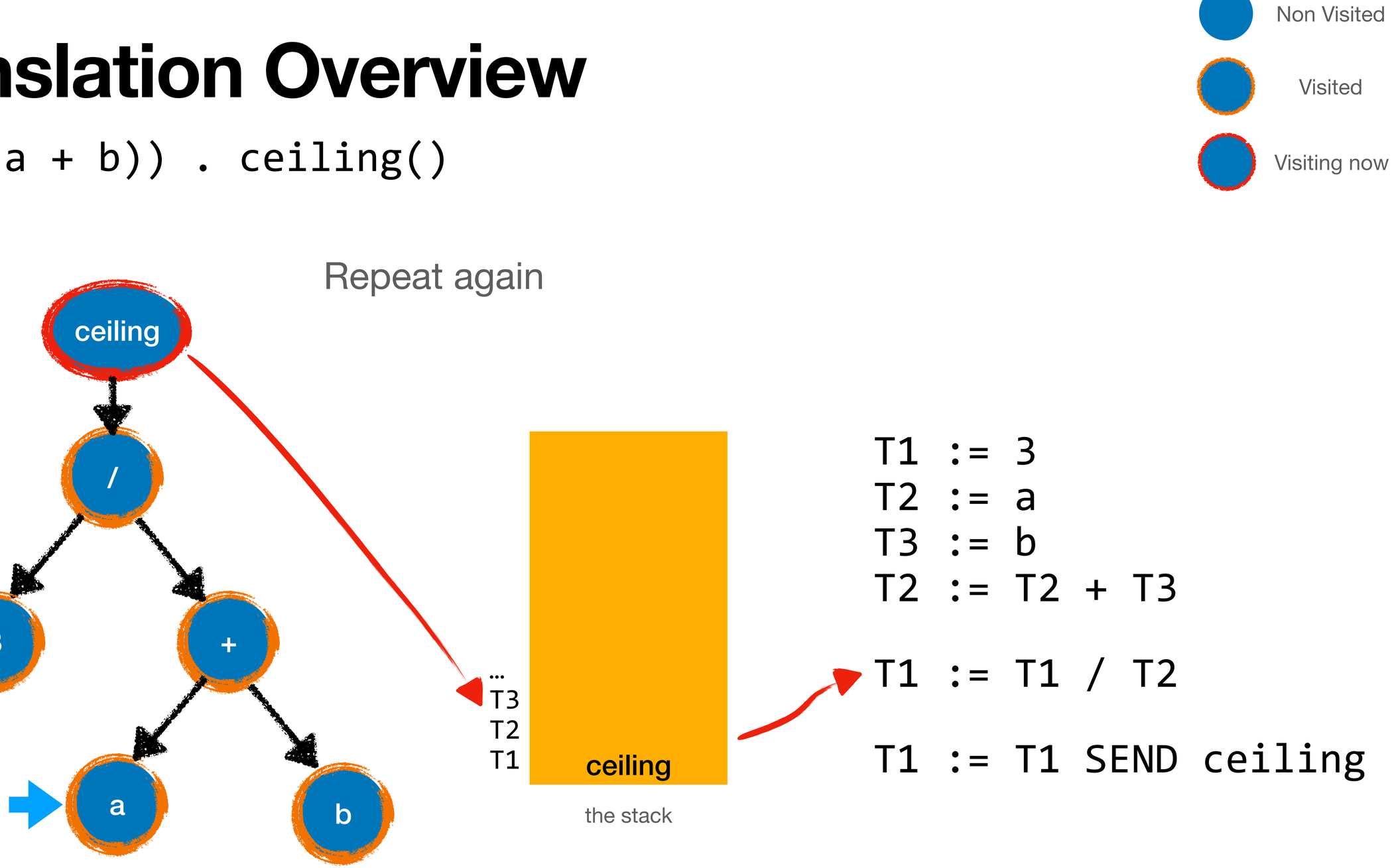
Translation Overview

```
(3 / (a + b)) . ceiling()
```



Translation Overview

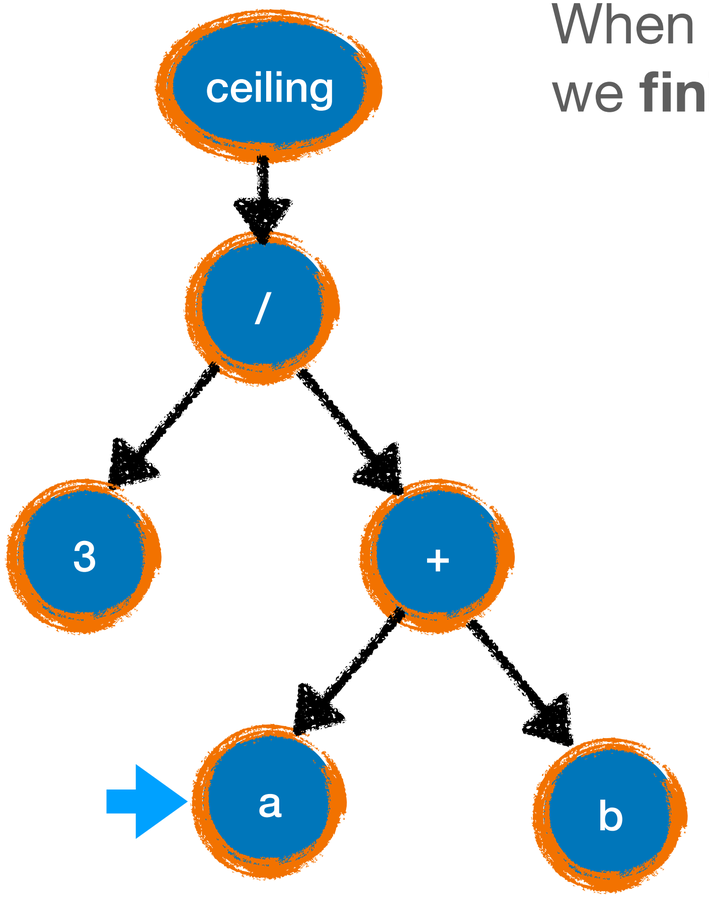
```
(3 / (a + b)) . ceiling()
```



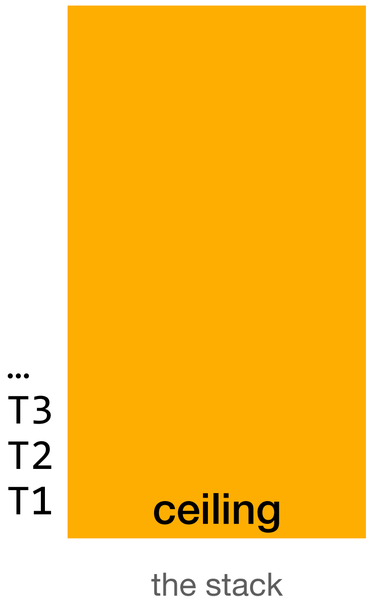
Translation Overview

```
(3 / (a + b)) . ceiling()
```





When the root is visited,
we **finished**!

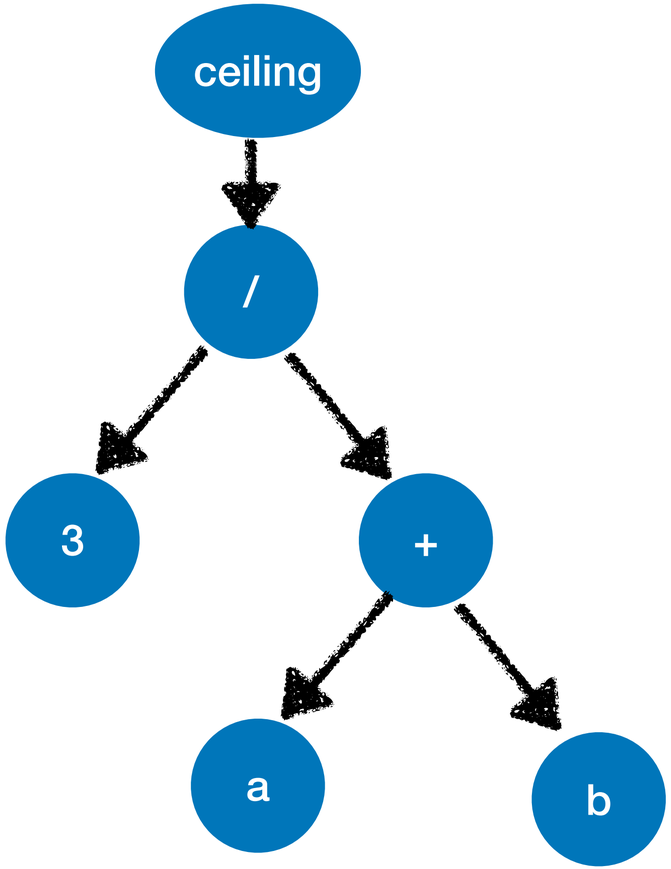



```
T1 := 3
T2 := a
T3 := b
T2 := T2 + T3

T1 := T1 / T2

T1 := T1 SEND ceiling
```

Adapting to Physical Registers






```
RAX := 3
RBX := a
RDI := b
RBX := RBX + RDI

RAX := RAX / RBX

RAX := RAX SEND ceiling
```

Map to physical registers

A Simple Register Allocation Algorithm

- Recursive implementation + register pool
- **Insights**
  - A new register is needed for each left hand side (`R1 := ...`)
  - When recursing, each node visited returns the register used
  - Then the parent uses those registers as operands
  - After using those, we return the register to the pool

Simple Register Allocation Pseudocode

```
BasicTranslator >> visit: aNode
  | operands resultRegister |
  operands := aNode operands collect: [ :op | self visit: op ].
  resultRegister := registerPool takeRegister.
  self
    generateInstruction: aNode
    on: resultRegister ops: operands.
  operands do: [ :reg | registerPool returnRegister: reg ].
  ^ resultRegister
```

Simple Register Allocation Pseudocode

```
BasicTranslator >> visit: aNode
  | operands resultRegister |
  operands := aNode operands collect: [ :op | self visit: op ].
  resultRegister := registerPool takeRegister.
  self
    generateInstruction: aNode
    on: resultRegister ops: operands.
  operands do: [ :reg | registerPool returnRegister: reg ].
  ^ resultRegister
```

Generate child instructions (post-order)

Simple Register Allocation Pseudocode

```
BasicTranslator >> visit: aNode
  | operands resultRegister |
  operands := aNode operands collect: [ :op | self visit: op ].
  resultRegister := registerPool takeRegister.
  self
    generateInstruction: aNode
    on: resultRegister ops: operands.
  operands do: [ :reg | registerPool returnRegister: reg ].
  ^ resultRegister
```

Get a free register from the pool

Simple Register Allocation Pseudocode

```
BasicTranslator >> visit: aNode
  | operands resultRegister |
  operands := aNode operands collect: [ :op | self visit: op ].
  resultRegister := registerPool takeRegister.
  self
    generateInstruction: aNode
    on: resultRegister ops: operands.
  operands do: [ :reg | registerPool returnRegister: reg ].
  ^ resultRegister
```

Generate the instruction with those reserved registers

Simple Register Allocation Pseudocode

```
BasicTranslator >> visit: aNode
  | operands resultRegister |
  operands := aNode operands collect: [ :op | self visit: op ].
  resultRegister := registerPool takeRegister.
  self
    generateInstruction: aNode
    on: resultRegister ops: operands.
  operands do: [ :reg | registerPool returnRegister: reg ].
  ^ resultRegister
```

Return the operand registers, we consumed them

Simple Register Allocation Pseudocode

```
BasicTranslator >> visit: aNode
  | operands resultRegister |
  operands := aNode operands collect: [ :op | self visit: op ].
  resultRegister := registerPool takeRegister.
  self
    generateInstruction: aNode
    on: resultRegister ops: operands.
  operands do: [ :reg | registerPool returnRegister: reg ].
  ^ resultRegister
```

Return this instruction's register to the parent instruction

Open issues

- What if you run out of registers? We will see spilling on a following lecture.

- What if I want to reuse a computation? If I return the register, it will be *discarded.* We will see code reuse algorithms later ;)

Conclusion

- Generating register based code requires allocating registers
- Register allocation can be though in different ways
  - a simple register pool
  - a mapping between stack slots and registers

- Then the algorithm can be extended for physical registers

Material

- Princeton Lecture: https://www.cs.princeton.edu/courses/archive/spring16/cos320/lectures/12-RegisterAllocation.pdf

- The Sethi–Ullman algorithm
  - https://doi.org/10.1145/321607.321620

- The dragon book. Aho and Ullman